

Санкт-Петербургский Политехнический Университет Петра Великого
Институт Информационных Технологий и Управления
Кафедра Информационных и Управляющих Систем

КУРСОВАЯ РАБОТА

по дисциплине «ЭВМ и периферийные устройства»

Выполнила
студентка гр. 23504/3



Е.А. Кабешова

Руководитель



С.А. Молодяков

Санкт-Петербург
2014

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
ЗАДАНИЕ №1	4
1.1. Формулировка	4
1.2. Текст программы	4
1.3. Пояснение работы программы	8
1.4. Скриншоты	9
ЗАДАНИЕ №2	10
2.1. Формулировка	10
2.2. Текст программы	10
2.3. Пояснение работы программы	14
2.4. Скриншоты	15

ВВЕДЕНИЕ

Ассемблер – низкоуровневый машинно-ориентированный язык программирования. Реализация языка зависит от типа процессора и определяется архитектурой вычислительной системы. Ассемблер позволяет напрямую работать с аппаратурой компьютера. Программа на языке ассемблера включает в себя набор команд, которые после трансляции преобразуются в машинные команды.

Также программы, написанные на языке ассемблера, требуют значительно меньшего объема памяти и времени выполнения. Знание программистом языка ассемблера и машинного кода дает ему понимание архитектуры машины. Несмотря на то, что большинство специалистов в области программного обеспечения разрабатывают программы на языках высокого уровня, наиболее мощное и эффективное программное обеспечение полностью или частично написано на языке ассемблера.

В задании №1 рассмотрена работа очереди. Очередь – структура данных с дисциплиной доступа к элементам «первый пришёл – первый вышел» (FIFO, First In – First Out). Добавление элемента возможно лишь в конец очереди, выборка происходит только из начала очереди, при этом выбранный элемент удаляется.

В задании №2 рассмотрена мультизадачность. Мультипланирование предназначено для повышения пропускной способности вычислительной системы путем более равномерной и полной загрузки ее оборудования, в первую очередь процессора.

Программы, рассмотренные в курсовой работе, были написаны в приложении ASMTTool Professional Edition.

ЗАДАНИЕ №1

1.1. Формулировка

Разработать набор процедур для работы с очередью, а именно: включение нового элемента; выборка очередного элемента (со сдвигом очереди).

Разработать демо-программу.

1.2. Текст программы

```
DOSSEG
.MODEL small
.386
.STACK 100h
.DATA

fifo DB 10 dup ('-') ;очередь

fifo_size DW 8 ;размер очереди
fifo_end DW 0 ;указывает на конец очереди

wr_num DW 0 ;количество записываемых элементов
rd_num DW 0 ;количество читаемых элементов

f_msg DB 'FIFO: ',10,13,'$'
input_msg DB 'Enter number of input elements:',10,13,'$'
elem_msg DB 'Enter elements of array:',10,13,'$'
output_msg DB 'Enter number of output elements:',10,13,'$'
empty_msg DB 'FIFO is empty!',10,13,'$'
full_msg DB 'FIFO is full!',10,13,'$'
excess_msg DB 'Your number become max of avilable',10,13,'$'
retry_msg DB 'It is very big number, try again!',10,13,'$'

.CODE

;-----
;процедура выводющая содержимое очереди
;-----

print_fifo proc
;вывод надписи "FIFO: "
    mov ah,9
    mov dx,offset f_msg
    int 21h
;вывод содержимого очереди
    xor si,si
    lea bx,fifo
    mov cx,fifo_size
print:
    mov dl,[bx+si]
    mov ah,2
    int 21h
    inc si
loop print
    mov al,10 ;перенос строки
    int 29h
    int 29h
ret
print_fifo endp
```

```

;-----
;процедура включения новых элементов в очередь
;-----

write_fifo proc
;проверка на заполненность очереди
    mov ax,fifo_size
    cmp fifo_end,ax
    je full_label
;вывод сообщения о количестве заполняемых элементов
    mov ah,9
    mov dx,offset input_msg
    int 21h
;ввод количества заполняемых элементов
    mov ah,0 ;ввод информации в консоль
    int 16h
    mov byte ptr wr_num,al ;запись числа в переменную
    sub word ptr wr_num,30h ;преобразование ASCII кода в число
    mov dl,al ;вывод введенного числа
    mov ah,2
    int 21h
    mov al,10 ;перенос строки
    int 29h
;проверка на превышение значения wr_num
    mov bx,fifo_end
    mov ax,fifo_size
    sub ax,bx
    mov bx,wr_num
    cmp bx,ax
    jb excess_label
;записываем в wr_num разность fifo_size и fifo_end
    mov ah,9
    mov dx,offset excess_msg
    int 21h
    mov bl,byte ptr fifo_end
    mov al,byte ptr fifo_size
    sub al,bl
    mov byte ptr wr_num,al
    add al,30h
    int 29h
    mov al,10
    int 29h
    excess_label:
;вывод сообщения о заполнении очереди
    mov ah,9
    mov dx,offset elem_msg
    int 21h
;заполнение очереди
    xor si,si
    add si,fifo_end
    lea bx,fifo
    mov cx,wr_num
input:
    mov ah,0
    int 16h
    mov [bx+si],al
    inc si
    mov dl,al
    mov ah,2
    int 21h
loop input
;обновляем значение fifo_end
    mov bx,wr_num
    add fifo_end,bx

```

```

        jmp over_full
;вывод сообщения о заполненности очереди
full_label:
        mov ah,9
        mov dx,offset full_msg
        int 21h
over_full:
        mov al,10 ;перенос строки
        int 29h
ret
write_fifo endp

;-----
;процедура чтения элементов из очереди
;-----

read_fifo proc
;проверка на пустоту очереди
        cmp fifo_end,0
        je empty_label
try_again:
;вывод сообщения о количестве читаемых элементов
        mov ah,9
        mov dx,offset output_msg
        int 21h
;ввод количества выводимых элементов
        mov ah,0 ;ввод информации в консоль
        int 16h
        mov byte ptr rd_num,al ;запись числа в переменную
        sub word ptr rd_num,30h ;преобразование ASCII кода в число
;проверка на превышение значения rd_num
        mov bx,rd_num
        cmp bx,fifo_end
        jae retry_label
        mov dl,al ;вывод введенного числа
        mov ah,2
        int 21h
        mov al,10 ;перенос строки
        int 29h
;чтение из очереди
        xor si,si
        lea bx,fifo
        mov cx,rd_num
output:
        mov dl,[bx+si]
        mov ah,2
        int 21h
        inc si
loop output
;сдвиг элементов
        xor di,di
        xor si,si
        add si,rd_num
        lea bx,fifo
        mov cx,fifo_end
        sub cx,rd_num
moving:
        mov al,[bx+si]
        mov [bx+di],al
        inc di
        inc si
loop moving
        mov fifo_end,di

```

```

;зачистка
    mov si,fifo_end
    mov cx,fifo_size
    sub cx,fifo_end
erasing:
    mov byte ptr [bx+si], '-'
    inc si
loop erasing
    jmp over_empty
;вывод сообщения о том, что очередь пуста
empty_label:
    mov ah,9
    mov dx,offset empty_msg
    int 21h
    jmp over_empty
;вывод сообщения о том, что введено большое число
retry_label:
    mov ah,9
    mov dx,offset retry_msg
    int 21h
    jmp try_again
over_empty:
    mov al,10 ;перенос строки
    int 29h
ret
read_fifo endp

```

```

;-----
;начало основной программы
;-----

```

```

start:

    mov ax,@data
    mov ds,ax
    xor ax,ax

    call write_fifo
    call print_fifo
    call read_fifo
    call print_fifo
    call write_fifo
    call print_fifo
    call read_fifo
    call print_fifo

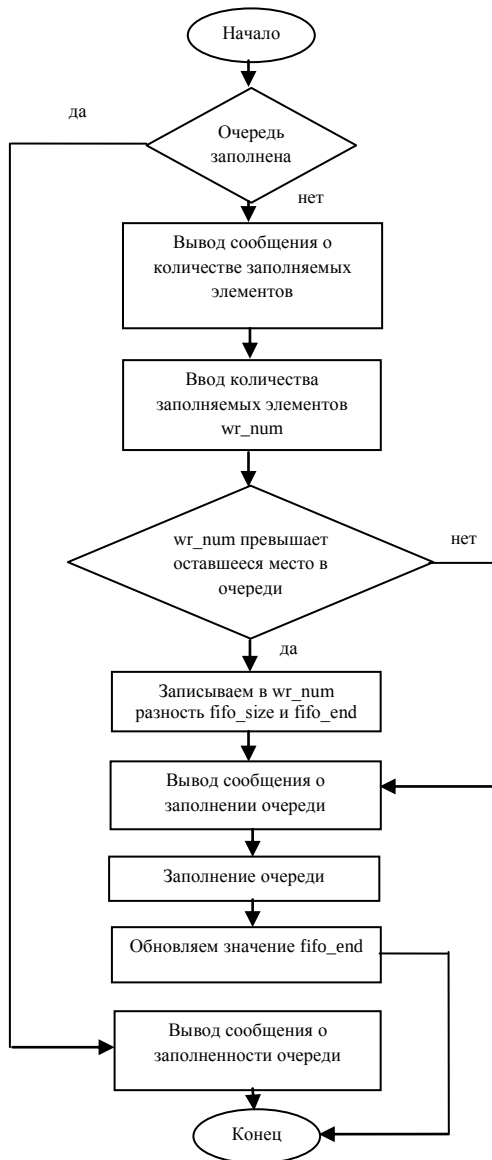
    mov ax,4c00h
    int 21h

END start

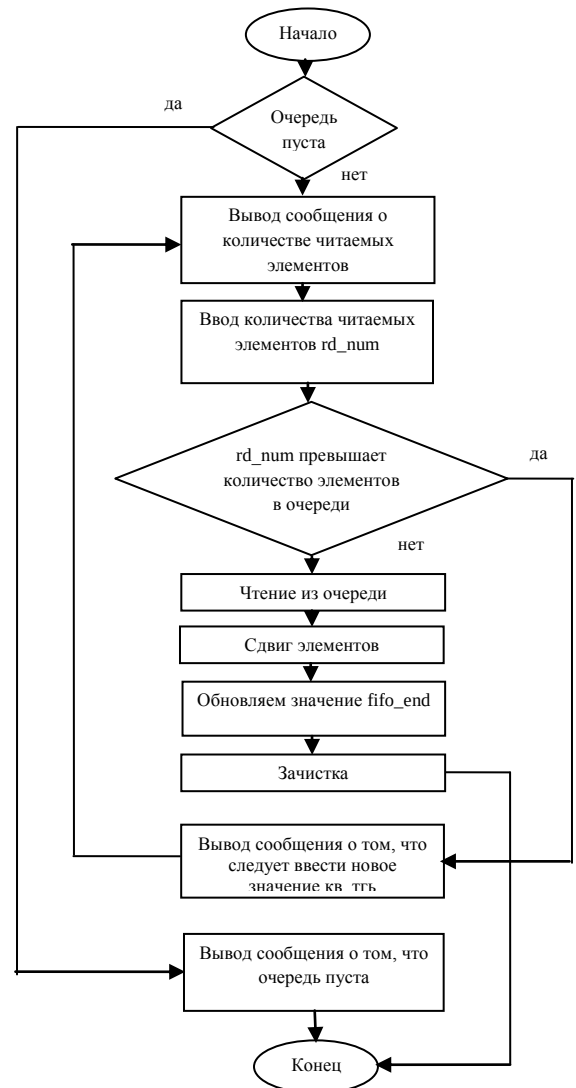
```

1.3. Пояснение работы программы

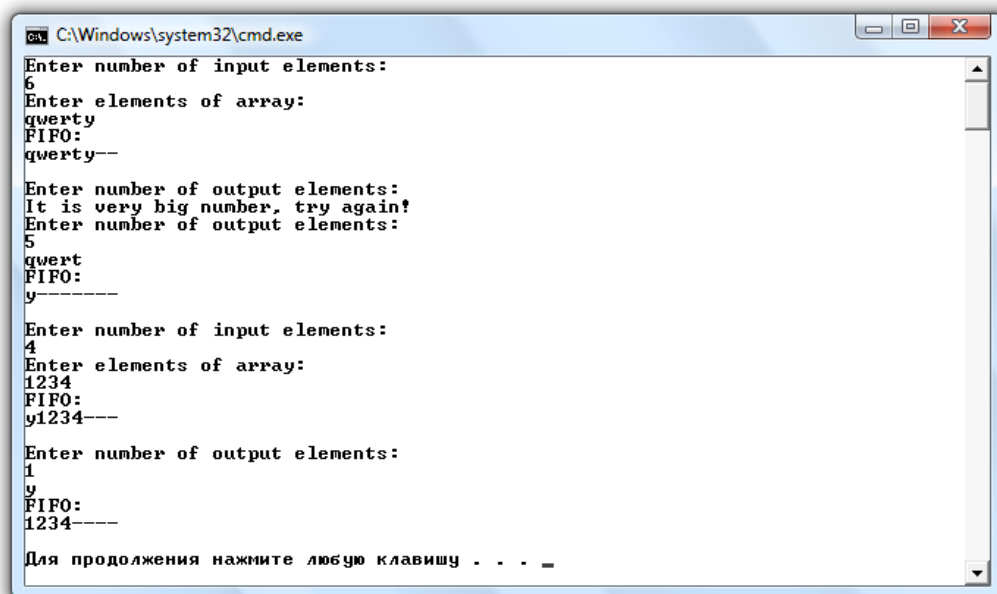
Процедура записи в очередь



Процедура чтения из очереди



1.4. Скриншоты



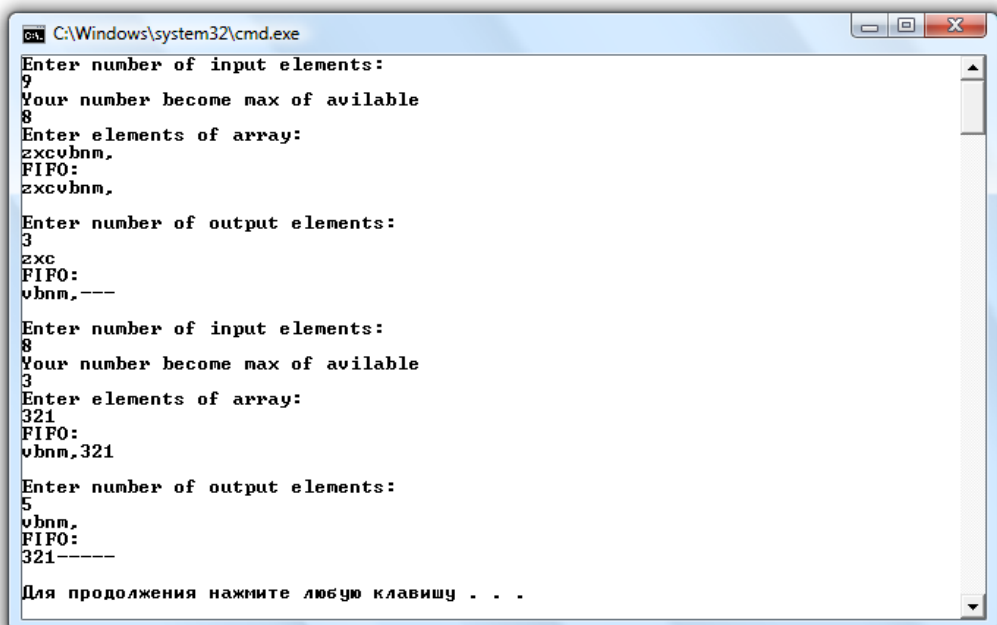
```
C:\Windows\system32\cmd.exe
Enter number of input elements:
6
Enter elements of array:
qwerty
FIFO:
qwerty--

Enter number of output elements:
It is very big number, try again!
Enter number of output elements:
5
qwerty
FIFO:
y-----

Enter number of input elements:
4
Enter elements of array:
1234
FIFO:
y1234---

Enter number of output elements:
1
y
FIFO:
1234----

Для продолжения нажмите любую клавишу . . .
```



```
C:\Windows\system32\cmd.exe
Enter number of input elements:
9
Your number become max of available
8
Enter elements of array:
zxcvbnm,
FIFO:
zxcvbnm,

Enter number of output elements:
3
zxc
FIFO:
vbnm,---

Enter number of input elements:
8
Your number become max of available
3
Enter elements of array:
321
FIFO:
vbnm,321

Enter number of output elements:
5
vbnm,
FIFO:
321-----

Для продолжения нажмите любую клавишу . . .
```

ЗАДАНИЕ №2

2.1. Формулировка

Организовать мультизадачную среду (3 и более задач) с выводом на экран номера задачи. Переключение происходит по алгоритму адаптивного планирования.

2.2. Текст программы

```
DOSSEG
.MODEL tiny
.386
.STACK 100h
.DATA

context_struct struc ;структура для хранения регистров
    rax    DW    ?
    rbx    DW    ?
    rcx    DW    ?
    rdx    DW    ?
    rds    DW    ?
    res    DW    ?
context_struct ends

context1 context_struct <> ;контекст 1-й задачи
context2 context_struct <> ;контекст 2-й задачи
context3 context_struct <> ;контекст 3-й задачи

p1    DW    3 ;приоритет 1-й задачи
p2    DW    2 ;приоритет 2-й задачи
p3    DW    1 ;приоритет 3-й задачи

handling_number    DW    32 ;количество обработок
msg DB 'Press any key to change procedure',10,13,'$'

ORG 100h ;увеличивает смещение последующих команд на 256

old_handler    DD    0 ;старый вектор

.CODE

proc1 proc
    ;загружаем регистры из контекста
    mov ax, context1.rax
    mov bx, context1.rbx
    mov cx, context1.rcx
    mov dx, context1.rdx
    mov ds, context1.rds
    mov es, context1.res

    mov al,'1' ;для вывода '1' с помощью int 29h

    ;после работы сохраняем регистры в контекст
    mov context1.rax, ax
    mov context1.rbx, bx
    mov context1.rcx, cx
    mov context1.rdx, dx
    mov context1.rds, ds
    mov context1.res, es
ret
proc1 endp
```

```

proc2 proc
    ;загружаем регистры из контекста
    mov ax, context2.rax
    mov bx, context2.rbx
    mov cx, context2.rcx
    mov dx, context2.rdx
    mov ds, context2.rds
    mov es, context2.res

    mov al,'2' ;для вывода '2' с помощью int 29h

    ;после работы сохраняем регистры в контекст
    mov context2.rax, ax
    mov context2.rbx, bx
    mov context2.rcx, cx
    mov context2.rdx, dx
    mov context2.rds, ds
    mov context2.res, es
ret
proc2 endp

proc3 proc
    ;загружаем регистры из контекста
    mov ax, context3.rax
    mov bx, context3.rbx
    mov cx, context3.rcx
    mov dx, context3.rdx
    mov ds, context3.rds
    mov es, context3.res

    mov al,'3' ;для вывода '3' с помощью int 29h

    ;после работы сохраняем регистры в контекст
    mov context3.rax, ax
    mov context3.rbx, bx
    mov context3.rcx, cx
    mov context3.rdx, dx
    mov context3.rds, ds
    mov context3.res, es
ret
proc3 endp

new_handler proc far
;алгоритм адаптивного планирования

;если приоритет 1-й задачи равен 0, то устанавливаем стандартные значения
    mov di,p1
    cmp di,0
    jne again
    mov di,3
    mov p1,di
    mov di,2
    mov p2,di
    mov di,1
    mov p3,di

;находим задачу с наибольшим приоритетом
again:
    mov si,p1
    mov di,p2
    cmp si,di
    jb m1
    ;p1 > p2

```

```

        mov dx,si
        jmp m2
m1:
        ;p1 < p2
        mov dx,di
m2:
        mov si,p3
        cmp si,dx
        jb m3
        ;p3 > (p1|p2)
        mov dx,si
m3:
        ;p3 < (p1|p2)

; после выбора задачи уменьшаем ее приоритет на 1
        cmp dx,si
        je l3
        cmp dx,di
        je l2
        mov si,p1
        cmp dx,si
        je l1

l1: mov si,1
    mov di,p1
    dec di
    mov p1,di
    jmp l4
l2: mov si,2
    mov di,p2
    dec di
    mov p2,di
    jmp l4
l3: mov si,3
    mov di,p3
    dec di
    mov p3,di
l4:

; выход после handling_number обработок
        mov cx,handling_number
        dec cx
        mov handling_number,cx
        cmp cx,0
        jl exit ;<

repeat:
; выполняем процедуру с наивысшим приоритетом
        pusha ; загружаем в стек регистры общего назначения
        cmp si,1
        je t1
        cmp si,2
        je t2
        cmp si,3
        je t3

t1: call proc1
    jmp print_task
t2: call proc2
    jmp print_task
t3: call proc3

```

```

print_task:
;вывод слова 'task'
    push ax
    mov al,'t'
    int 29h
    mov al,'a'
    int 29h
    mov al,'s'
    int 29h
    mov al,'k'
    int 29h
    mov al,' '
    int 29h
    pop ax

    int 29h ;вывод '1', '2' или '3' из регистра al
    mov al,10 ;перенос строки
    int 29h
    pora ;выгружаем из стека регистры общего назначения

;сброс контроллера прерываний
    mov al,20h
    out 20h,al

;проверка нажата ли клавиша
    mov ah,1
    int 16h
    jz repeat ;повторяем, если не нажали

iret
new_handler endp

main:
;вывод приветственного сообщения
    mov dx,offset msg
    add dx,100h
    mov ah,9
    int 21h

;сохраним текущий вектор прерывания с помощью int 21h
;вход: AH = 35h, AL = номер вектора прерывания
;выход: ES:BX = адрес программы обработки прерывания
;в BX заносится содержимое адреса 0:(4*AL), в ES - 0:(4*AL+2)
    mov ah,35h
    mov al,09h
    int 21h
    mov word ptr old_handler+2,es
    mov word ptr old_handler,bx

;установим новый вектор прерывания с помощью int 21h
;вход: AH = 25h, AL = номер прерывания (от клавиатуры 09h)
;DS:DX = вектор прерывания: адрес программы обработки прерывания
    push cs
    pop ds
    mov dx,offset new_handler
    mov ah,25h
    mov al,09h
    int 21h

```

```

;сохраняем регистры во все контексты
mov context1.rax, ax
mov context1.rbx, bx
mov context1.rcx, cx
mov context1.rdx, dx
mov context1.rds, ds
mov context1.res, es

mov context2.rax, ax
mov context2.rbx, bx
mov context2.rcx, cx
mov context2.rdx, dx
mov context2.rds, ds
mov context2.res, es

mov context3.rax, ax
mov context3.rbx, bx
mov context3.rcx, cx
mov context3.rdx, dx
mov context3.rds, ds
mov context3.res, es

infinity: jmp infinity ;бесконечный цикл

exit:
;вернуться к старому обработчику прерывания (заполнение DS:DX из old)
lds dx,old_handler
mov ah,25h ;функция заполнения вектора
mov al,09h ;номер вектора
int 21h

mov ah,4ch ;функция DOS завершения программы
int 21h ;завершить программу

END main

```

2.3. Пояснение работы программы

Мультизадачная среда позволяет переключаться от одной запущенной подпрограммы к другой. Для того чтобы организовать такую среду, можно заменить какое-нибудь прерывание. В нашей задаче мы заменяем стандартное прерывание от клавиатуры (int 9h) на свое собственное, которое будет запускать нужную нам подпрограмму.

Контекст задач, в котором хранятся регистры ax, bx, cx, dx, ds es представляет собой структуру context_struct.

Алгоритм адаптивного планирования подразумевает, что у каждой подпрограммы есть свой приоритет. Запускается подпрограмма с большим приоритетом, после чего ее приоритет падает на 1. В нашей задаче у первой подпрограммы приоритет 3, у второй – 2, у третьей – 1. Таким образом, смена подпрограмм осуществляется по алгоритму: 121321 и т.д. Когда приоритет первой задачи становится равным 0, то приоритет всех задач становится стандартным, который был задан в начале.

2.4. Скриншоты

